Daffodil International University

Faculty of Science and Information Technology

Department of Computing and Information System (CIS)

Lab Manual

Course Title	Data Structure Lab
Course Code	CIS 122L
Course Type	Lab
Course Credit	1
Contact Hours	90 Minutes
Level/Term	Level 1 Term 2
Prerequisite(s)	CIS 115L
Instructor Information	ISRAFIL Lecturer, Convener, Exam Committee, Dept. of CIS, FSIT Department of Computing and Information System (CIS) Daffodil International University **Mobile:** +8801639701573 (WhatsApp)

Table of Contents

Course Objectives 4
Course Outcomes 4
Preferred Programming Language(S) 4
Preferred Programming Tool(S) 4
Text/Reference Book(s) 4
Marks Distribution 5
Term Final Examination 5
Week 01 6
A program to insert an element into an array at a specific position 6
Week 02 7
A program to delete and update an element in an array at a specific position 7
Week 03 9
A program to split an array into two arrays based on odd and even numbers. 9
Week 04 10
A program for merging two sorted arrays into one sorted array. 10
Week 05 12
A program to insert an element into a stack (PUSH) and remove an element from a stack (POP) 12
Week 06 14
A program to insert an element into a queue (Enqueue) and delete an element from a queue (Dequeue) 14
Week 07 16
A program for traversing a linked list. 16
Week 08 18
A program for searching in a singly linked list. 18
Week 09 20
A program for splitting a singly linked list based on a specified position/condition 20
Week 10 24
A program to merge two simple linked lists into a single linked list 24
Week 11 26
A program for inserting nodes at the beginning, middle, and end of a simple linked list. 26

Week 12 28
A program for deleting nodes from the beginning, middle, and end of a linked list 28
Week 13..... 30
A program for inserting nodes into a binary search tree 30
Week 14 32
A program for Linear and Binary Searching. 32
Week 15..... 34
A program for different sorting operation (Insertion, Selection and Bubble) 34
Final Challenge Problems..... 37

Course Objectives

- ➔ To write and execute programs in C to solve problems using data structures such as arrays, linked lists, stacks, queues.
- ➔ To write and execute programs in C to solve problems using data structures such as trees, graphs, hash tables and search trees.
- ➔ To write and execute write programs in C to implement various sorting and searching methods.

Course Outcomes

- ➔ Graduates will be able to identify the appropriate data structure for a given problem.
- ➔ Graduates will be able to design programs for solving problems using different data structures.
- ➔ Graduates can solve problems using trees, graphs and hash tables addressing various issues.
- ➔ Graduates will have the ability to effectively use compilers including library functions, debuggers and trouble shooting.

Preferred Programming Language(S)

- ➔ C
- ➔ C++

Preferred Programming Tool(S)

- ➔ Code Blocks

Text/Reference Book(s)

- ➔ Edward M. Reingold & Wilfred J. Hansen, ***Data Structures***, Addison Wesley School, 1998.
- ➔ Seymour Lipschutz, ***Schaum's outline of theory and problems of data structures***, McGraw-Hill, 1986.

Marks Distribution

Assessment No.	Assessment Name	Percentage
1	Class Attendance	10%
2	Lab Performance	25%
3	Semester Assignment	25%
4	Term Final Examination	40%
Total		100%

Term Final Examination

There will be a one-hour final examination. Different types of questions will be included such as MCQ, writing programs etc.

Week 01

A program to **insert** an element into an array at a specific position.

Program

```
#include<stdio.h>
main()
{
    int n,i,j,v;
    printf("Enter the number of element:\n");
    scanf("%d",&n);
    printf("Input %d element:\n",n);
    int a[n+1];
    for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    printf("Insert array position:\n");
    scanf("%d",&i);
    printf("Insert array value:\n");
    scanf("%d",&v);
    if(i>=n)
    a[n]=v;
    else
{
    for(j=n;j>i;j--)
    a[j]=a[j-1];
    a[j]=v;
}
    printf("The present array is:\n");
    for(j=0;j<n+1;j++)
    printf("%d\n",a[j]);
}
```

Sample Output:

```
Enter the number of element:
4
Input 4 element:
10 20 30 40
Insert array position:
2
Insert array value:
800
The present array is:
10
20
800
30
40
```

Practice Problems

I. Write a program to insert multiple elements at specific positions in an array.
II. Write a program to insert an element into an array in ascending order (maintain sorted order).

Week 02

A program to **delete** and **update** an element in an array at a specific position.

Program (Delete)

```
#include<stdio.h>
int main()
{
    int i,c,n;
    printf("Enter the number of element an array:\n");
    scanf("%d",&n);
    int ara[n];
    printf("Enter %d elements \n",n);
    for(c=0;c<n;c++)
    scanf("%d",&ara[c]);
    printf("Enter the position to delete an element\n");
    scanf("%d",&i);
    if(i>=n)
    printf("delete is not possible\n");
    else
{
    for(c=i+1;c<n;c++)
    ara[c-1]=ara[c];
    printf("Resultant array is\n");
    for(c=0;c<n-1;c++)
    printf("% d",ara[c]);
}
    return 0;
}
```

Sample Output:

```
Enter the number of element an array:
5
Enter 5 elements
10 20 30 40 50
Enter the position to delete an element
3
Resultant array is
 10 20 30 50
```

Program (Update)

```
#include<stdio.h>
main()
{
    int i,j,v,n;
    printf("Enter the number of array element \n");
    scanf("%d",&n);
    int a[n];
    printf("Enter the value\n");
    for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    printf("Enter the update value\n");
    scanf("%d",&v);
  printf("Enter the position\n");
    scanf("%d",&i);
    if(i>=n)
    printf("update is not possible\n");
    else
{
    a[i]=v;
}
    printf("The update array is\n");
    for(i=0;i<n;i++)
{
    printf("%d \n",a[i]);
}
}
```

Sample Output:

```
Enter the number of array element
4
Enter the value
23 45 65 12
Enter the update value
600
Enter the position
1
The update array is
23
600
65
12
```

Practice Problems

I. Write a program to delete all occurrences of a specific value from an array.
II. Write a program to update all even elements in an array by adding 5.

Week 03

A program to **split** an array into two arrays based on odd and even numbers.

Program

```
#include<stdio.h>
main()
{
   int i,j,k,n,even=0,odd=0;
   printf("Enter the number of element of
the array A[]:\n");
   scanf("%d",&n);
   int a[n],b[n],c[n];
   printf("\nEnter %d elements:\n",n);
   for(i=0; i<n; i++)
      scanf("%d",&a[i]);
   if(n==0)
      printf("Array is Empty\n");
   i=0,j=-1,k=-1;
   for(i=0; i<n; i++)
   {
      if(a[i]%2==0)
      {
         j=j+1;
         b[j]=a[i];
         even++;
      }
      else
      {
         k=k+1;
         c[k]=a[i];
         odd++;
      }
   }

   printf("\nAfter split the even array B[]
is:\n");
   for(j=0; j<even; j++)
      printf("%d ",b[j]);
   printf("\nAfter split The odd array C[]
is:\n");
   for(k=0; k<odd; k++)
      printf("%d ",c[k]);
}
```

Sample Output:

```
Enter the number of element of the array A[]:
5

Enter 5 elements:
14 77 83 34 98

After split the even array B[] is:
14 34 98
After split The odd array C[] is:
77 83
```

Practice Problems

I. Write a program to split an array into multiples of 3 and non-multiples of 3.
II. Write a program to split an array into positive and negative numbers.

Week 04

A program for **merging** two sorted arrays into one sorted array.

Program

```
#include<stdio.h>
main()
{
    int i,j,k,m,n,p,r;
    printf("Enter the number of element of array B:\n");
    scanf("%d",&m);
    int b[m];
    printf("Enter the sorted %d element:\n",m);
    for(j=0;j<m;j++)
    scanf("%d",&b[j]);
    printf("Enter the number of element of array C:\n");
    scanf("%d",&p);
    int c[p];
    printf("Enter the sorted %d element:\n",p);
    for(k=0;k<p;k++)
    scanf("%d",&c[k]);
    int a[m+p];
    i=-1,j=0,k=0;
    M:
    if(j<m&&k<p)
{
    if(b[j]>c[k])
{
    i++;
    a[i]=c[k];
    k++;
}
    else
{
    i++;
    a[i]=b[j];
    j++;
}
    goto M;
}
    if(j<m)
{
    for(r=j;r<m;r++)
{
    i++;
    a[i]=b[r];
}
}
    else
{
    for(r=k;r<p;r++)
{
    i++;
    a[i]=c[r];
}
}
    printf("The merge sorted array of A is:\n");
    for(i=0;i<m+p;i++)
    printf("\n%d ",a[i]);
}
```

Sample Output:

```
Enter the number of element of array B:
3
Enter the sorted 3 element:
34 56 78
Enter the number of element of array C:
3
Enter the sorted 3 element:
12 35 38
The merge sorted array of A is:

12
34
35
38
56
78
```

Practice Problems

I. Write a program to merge two arrays and remove duplicate elements.
II. Write a program to merge three sorted arrays into one sorted array.

Week 05

A program to insert an element into a stack (**PUSH**) and remove an element from a stack (**POP**).

Program

```
#include<stdio.h>
main()
{
   int top=-1, InValue, i, j,n, m;
   printf("Enter the number of element (capacity) of Stack\n");
   scanf("%d",&n);
   int a[n], maxsize=n-1;
   for( ; ;)
   {
      printf("Enter 1 for Insert\nEnter 2 for Delete\nEnter 3 for Display\nEnter others
key for Exit\n");
      scanf("%d",&m);
      if(m==1)
      {
         if(top==maxsize)
            printf("Overflow\n\n");
         else
         {
            printf("Enter your insert value\n");
            scanf("%d",&InValue);
            top=top+1;
            a[top]=InValue;
            printf("Data successfully inserted in the stack\n\n");
         }
      }
else if(m==2)
      {
         if(top==-1)
            printf("Underflow\n");
         else
         {
            a[top]=NULL;
            top=top-1;
            printf("Data successfully deleted from the stack\n\n");
         }
      }

      else if(m==3)
```

```
        {
            printf("Stack values are\n");
            for(j=0; j<=top; j++)
                printf("%d\n",a[j]);
        }
        else
            exit(0);
    }
}
```

Sample Output:

```
Enter the number of element (capacity) of Stack
4
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter others key for Exit
1
Enter your insert value
500
Data successfully inserted in the stack

Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter others key for Exit
1
Enter your insert value
600
Data successfully inserted in the stack

Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter others key for Exit
3
Stack values are
500
600
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter others key for Exit
2
Data successfully deleted from the stack

Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter others key for Exit
3
Stack values are
500
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter others key for Exit
```

Practice Problems

I. Write a program to implement stack using functions for PUSH, POP, and DISPLAY.
II. Write a program to reverse an array using a stack.

Week 06

A program to insert an element into a queue (**Enqueue**) and delete an element from a queue (**Dequeue**).

Program

```
#include<stdio.h>
int main()
{
   int n,m,value,j,front=-1,rear=-
1,count=0;
   printf("Enter the number of element of
Queue\n");
   scanf("%d",&n);
   int a[n], maxsize=n-1;
   for(j=0; j<=maxsize; j++)
      a[j]=0;
   for( ; ;)
   {
      printf("Enter 1 for Insert\nEnter 2
for Delete\nEnter 3 for Display\nEnter 0
for Exit\n");
      scanf("%d",&m);
      if(m==1)
      {
         printf("Enter your value:\n");
         scanf("%d",&value);
         if(count==n)
            printf("Queue is Overflow\n");
         else
         {
            if(rear==-1)
            {
               front=0;
               rear=0;
            }
            else if(rear==maxsize)
               rear=0;
            else
               rear=rear+1;

            a[rear]=value;
```

```
else if(m==2)
      {
         if(count==0)
            printf("Queue is Underflow\n");
         else
         {
            a[front]=0;
            count--;
            printf("Data successfully
deleted\n");
            if(count==0)
            {
               front=-1;
               rear=-1;
            }
            else if(front==maxsize)
               front=0;
            else
               front++;
         }
      }
      else if(m==3)
      {
         printf("The Queue values are\n");
         for(j=0; j<=maxsize; j++)
            printf("%d ",a[j]);
         printf("\n");
      }
      else
         exit(0);
   }
}
```

```
            count++;
            printf("Data successfully
Inserted\n");
        }
    }
```

Sample Output:

```
Enter the number of element of Queue
4
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter 0 for Exit
1
Enter your value:
10
Data successfully Inserted
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter 0 for Exit
1
Enter your value:
20
Data successfully Inserted
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter 0 for Exit
3
The Queue values are
10 20 0 0
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter 0 for Exit
2
Data successfully deleted
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter 0 for Exit
3
The Queue values are
0 20 0 0
Enter 1 for Insert
Enter 2 for Delete
Enter 3 for Display
Enter 0 for Exit
```

Practice Problems

I. Write a program to find the front and rear elements of a queue.
II. Write a program to count the total number of elements currently in the queue.

Week 07

A program for **traversing** a linked list.

Program

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
   int value;
   struct student *next;
};
typedef struct student node;
node *p,*q,*h;
int main()
{
   int x;
   h=0,q=0;
   for( ; ;)
   {
      printf("Enter your value (-99 to break)\n");
      scanf("%d",&x);
      if(x==-99)
         break;
      p=(node*)malloc(sizeof(node));
      if(h==0)
      {
         p->value=x;
         p->next=NULL;
         h=p;
         q=p;
      }
      else
      {
         p->value=x;
         p->next=NULL;
         q->next=p;
         q=p;
      }
   }
   printf("After Traverse, Linked List Values Are:\n");
   node *ptr=h;
   while(ptr!=NULL)
   {
      printf("%d\n",ptr->value);
      ptr=ptr->next;
   }
   return 0;
}
```

Sample Output:

```
Enter your value (-99 to break)
10
Enter your value (-99 to break)
20
Enter your value (-99 to break)
30
Enter your value (-99 to break)
-99
After Traverse, Linked List Values Are:
10
20
30
```

Practice Problems

I. Write a program to count the number of nodes in a linked list.
II. Write a program to find the sum of all node values in a linked list.

Week 08

A program for **searching** in a singly linked list.

Program

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
   int value;
   struct student *next;
};
typedef struct student node;
node *p,*q,*h;
int main()
{
   int x;
   h=0,q=0;
   for( ; ;)
   {
      printf("Enter your value(-99 to break)\n");
      scanf("%d",&x);
      if(x==-99)
         break;
      p=(node*)malloc(sizeof(node));
      if(h==0)
      {
         p->value=x;
         p->next=NULL;
         h=p;
         q=p;
      }
      else
      {
         p->value=x;
         p->next=NULL;
         q->next=p;
         q=p;
      }
   }
printf("Final linked list values are:\n");
   node *ptr=h;
   while(ptr!=NULL)
   {
      printf("%d\n",ptr->value);
      ptr=ptr->next;
   }

   int SV, value_found=0;
   printf("Enter your search value\n");
   scanf("%d",&SV);
   node *ptr1=h;
   while(ptr1!=NULL)
   {
      if(ptr1->value==SV)
      {
         value_found=1;
         break;
      }
      ptr1=ptr1->next;
   }
   if(value_found==1)
      printf("Value is found\n");
   else
      printf("Value is not found\n");

   return 0;
}
```

Sample Output:

```
Enter your value(-99 to break)
10
Enter your value(-99 to break)
20
Enter your value(-99 to break)
30
Enter your value(-99 to break)
-99
Final linked list values are:
10
20
30
Enter your search value
30
Value is found
```

Practice Problems

I. Write a program to search for a node by its position index.
II. Write a program to count occurrences of a specific value in the linked list.

Week 09

A program for **splitting** a singly linked list based on a specified position/condition.

Program (specified position)

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
   int value;
   struct student *next;
};
typedef struct student node;
node *p,*q,*h;
int main()
{
   int x;
   h=0,q=0;
   for( ; ;)
   {
      printf("Enter your value (-99 to break)\n");
      scanf("%d",&x);
      if(x==-99)
         break;
      p=(node*)malloc(sizeof(node));
      if(h==0)
      {
         p->value=x;
         p->next=NULL;
         h=p;
         q=p;
      }
      else
      {
         p->value=x;
         p->next=NULL;
         q->next=p;
         q=p;
      }
   }
printf("Final Linked List Values Are:\n");
   node *ptr=h;
   while(ptr!=NULL)
   {
      printf("%d\n",ptr->value);
      ptr=ptr->next;
   }

   node *h1,*h2;
   h1=h;
   h2=NULL;
   int i=1, position;
   printf("Enter your split position\n");
   scanf("%d",&position);
   ptr=h;
   while(i!=position)
   {
      i=i+1;
      ptr=ptr->next;
   }
   h2=ptr->next;
   ptr->next=NULL;

   printf("After Split, First Linked List Are:\n");
   while(h1!=NULL)
   {
      printf("%d\n",h1->value);
      h1=h1->next;
   }
   printf("After Split, Second Linked List Are:\n");
   while(h2!=NULL)
   {
      printf("%d\n",h2->value);
      h2=h2->next;
   }
   return 0;
}
```

Sample Output (specified position):

```
Enter your value (-99 to break)
10
Enter your value (-99 to break)
20
Enter your value (-99 to break)
30
Enter your value (-99 to break)
40
Enter your value (-99 to break)
-99
Final Linked List Values Are:
10
20
30
40
Enter your split position
3
After Split, First Linked List Are:
10
20
30
After Split, Second Linked List Are:
40
```

Program (specified condition)

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
   int value;
   struct student *next;
};
typedef struct student node;
node *p,*q,*h;
int main()
{
   int x;
   h=0,q=0;
   for( ; ;)
   {
```

```
if(ptr->value>10 && ptr->value%4==0)
    {
      if(h1==0)
      {
         p->value=ptr->value;
         p->next=NULL;
         h1=p;
         q1=p;
      }
      else
      {
         p->value=ptr->value;
         p->next=NULL;
         q1->next=p;
         q1=p;
      }
```

```
    printf("Enter your value (-99 to break)\n");
    scanf("%d",&x);
    if(x==-99)
      break;
    p=(node*)malloc(sizeof(node));
    if(h==0)
    {
      p->value=x;
      p->next=NULL;
      h=p;
      q=p;
    }
    else
    {
      p->value=x;
      p->next=NULL;
      q->next=p;
      q=p;
    }
  }
  printf("Final Linked List Values Are:\n");
  node *ptr=h;
  while(ptr!=NULL)
  {
    printf("%d\n",ptr->value);
    ptr=ptr->next;
  }

  node *h1,*h2,*q1,*q2;
  h1=0,q1=0;
  h2=0,q2=0;
  ptr=h;
  while(ptr!=NULL)
  {
    p=(node*)malloc(sizeof(node));
    }
    else
    {
      if(h2==0)
      {
        p->value=ptr->value;
        p->next=NULL;
        h2=p;
        q2=p;
      }
      else
      {
        p->value=ptr->value;
        p->next=NULL;
        q2->next=p;
        q2=p;
      }
    }
    ptr=ptr->next;
  }

  printf("After Split, First Linked List Are:\n");
  while(h1!=NULL)
  {
    printf("%d\n",h1->value);
    h1=h1->next;
  }

  printf("After Split, Second Linked List Are:\n");
  while(h2!=NULL)
  {
    printf("%d\n",h2->value);
    h2=h2->next;
  }
  return 0;
}
```

Sample Output (specified condition):

```
Enter your value (-99 to break)
12
Enter your value (-99 to break)
34
Enter your value (-99 to break)
56
Enter your value (-99 to break)
22
Enter your value (-99 to break)
8
Enter your value (-99 to break)
-99
Final Linked List Values Are:
12
34
56
22
8
After Split, First Linked List Are:
12
56
After Split, Second Linked List Are:
34
22
8
```

Practice Problems

I. Write a program to split a linked list into odd and even positioned nodes.
II. Write a program to separate nodes with values greater than 50 into a new linked list.

Week 10

A program to **merge** two simple linked lists into a single linked list.

Program

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
   int value;
   struct student *next;
};
typedef struct student node;
node *p1,*q1,*h1,*p2,*q2,*h2;
int main()
{
   int x;
   h1=0,q1=0,h2=0,q2=0;
   printf("First Linked List\n");
   for( ; ;)
   {
      printf("Enter your value (-99 to break)\n");
      scanf("%d",&x);
      if(x==-99)
         break;
      p1=(node*)malloc(sizeof(node));
      if(h1==0)
      {
         p1->value=x;
         p1->next=NULL;
         h1=p1;
         q1=p1;
      }
      else
      {
         p1->value=x;
         p1->next=NULL;
         q1->next=p1;
         q1=p1;
      }
   }
printf("Second Linked List\n");
   for( ; ;)
   {
      printf("Enter your value (-99 to break)\n");
      scanf("%d",&x);
      if(x==-99)
         break;
      p2=(node*)malloc(sizeof(node));
      if(h2==0)
      {
         p2->value=x;
         p2->next=NULL;
         h2=p2;
         q2=p2;
      }
      else
      {
         p2->value=x;
         p2->next=NULL;
         q2->next=p2;
         q2=p2;
      }
   }

   printf("Second linked list values are:\n");
   node *ptr2=h2;
   while(ptr2!=NULL)
   {
      printf("%d\n",ptr2->value);
      ptr2=ptr2->next;
   }

   node *h=h1;
   node *ptr=h1;
   while(ptr->next!=NULL)
      ptr=ptr->next;
   ptr->next=h2;
```

```
    printf("First linked list values are:\n");
    node *ptr1=h1;
    while(ptr1!=NULL)
    {
        printf("%d\n",ptr1->value);
        ptr1=ptr1->next;
    }
    printf("After merge, linked list values are:\n");
    node *ptr3=h;
    while(ptr3!=NULL)
    {
        printf("%d\n",ptr3->value);
        ptr3=ptr3->next;
    }
    return 0;
}
```

Sample Output:

```
Enter your value (-99 to break)
-99
First linked list values are:
12
8
56
Second Linked List
Enter your value (-99 to break)
45
Enter your value (-99 to break)
67
Enter your value (-99 to break)
87
Enter your value (-99 to break)
-99
Second linked list values are:
45
67
87
After merge, linked list values are:
12
8
56
45
67
87
```

Practice Problems

I. Write a program to merge two sorted linked lists into one sorted linked list.
II. Write a program to merge two linked lists alternately (node-by-node).

Week 11

A program for **inserting** nodes at the **beginning**, **middle**, and **end** of a simple linked list.

Program

```
#include<stdio.h>
struct student
{
   int value;
   struct student *Next;
};
typedef struct student node;
node *p,*q,*h,*p1;
main()
{
   int x,a,b,s;
   h=0;
   for(;;)
   {
      printf("Enter your value (-99 to break)\n");
      scanf("%d",&x);
      if(x==-99)
         break;
      else
      {

p=(node*)malloc(sizeof(node));
         if(h==0)
         {
            h=p;
            p->value=x;
            p->Next=NULL;
            q=p;
         }
         else
         {
            p->value=x;
            p->Next=NULL;
            q->Next=p;
            q=p;
p=(node*)malloc(sizeof(node));
         printf("Enter your insert option: 1 for 1st Insertion/2 for middle Insertion/ others for last Insertion\n");
         scanf("%d",&b);
         if(b==1)
         {
            p->Next=h;
            p->value=a;
            h=p;
            printf("Value is Successfully Inserted\n");
         }
         else if(b==2)
         {
            printf("Enter your search value:\n");
            scanf("%d",&s);
            p1=h;
            while(p1->value!=s)
               p1=p1->Next;
            p->Next=p1->Next;
            p1->Next=p;
            p->value=a;
            printf("Value is Successfully Inserted\n");
         }
         else
         {
            p->Next=NULL;
            p->value=a;
            p1=h;
            while(p1->Next!=0)
               p1=p1->Next;
            p1->Next=p;
            printf("Value is Successfully Inserted\n");
```

```
        }
      }
    }
    for(;;)
    {
      printf("\nMenu choice: 1 for Insertion/2 for Display/Others Key for Exit\n");
      scanf("%d",&x);
      if(x==1)
      {
        printf("Enter your insert value:\n");
        scanf("%d",&a);
        }
      }
      else if(x==2)
      {
        p1=h;
        while(p1!=NULL)
        {
          printf("\nVALUE %d | ADDRESS %d\n",p1->value,p1);
          p1=p1->Next;
        }
      }
      else
        break;
    }
}
```

Sample Output:

```
Enter your value (-99 to break)
10
Enter your value (-99 to break)
20
Enter your value (-99 to break)
30
Enter your value (-99 to break)
-99

Menu choice: 1 for Insertion/2 for Display/Others Key for Exit
2

VALUE  10   |  ADDRESS  7083088

VALUE  20   |  ADDRESS  7083120

VALUE  30   |  ADDRESS  7083152

Menu choice: 1 for Insertion/2 for Display/Others Key for Exit
1
Enter your insert value:
400
Enter your insert option: 1 for 1st Insertion/2 for middle Insertion/ others for last Insertion
2
Enter your search value:
20
Value is Successfully Inserted

Menu choice: 1 for Insertion/2 for Display/Others Key for Exit
2

VALUE  10   |  ADDRESS  7083088

VALUE  20   |  ADDRESS  7083120

VALUE  400   |  ADDRESS  7083184

VALUE  30   |  ADDRESS  7083152

Menu choice: 1 for Insertion/2 for Display/Others Key for Exit
```

Practice Problems

I. Write a program to insert a node at a given position index.
II. Write a program to create a sorted linked list by inserting nodes in sorted order.

Week 12

A program for **deleting** nodes from the **beginning**, **middle**, and **end** of a linked list.

Program

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
   int value;
   struct student *next;
};
int main()
{
   typedef struct student node;
   node *h=0,*PTR,*q=0,*ptr,*PTR1;
   int v,delete_value;
   for(;;)
   {
      printf("Enter your value (-99 to break)\n");
      scanf("%d",&v);
      if(v==-99)
         break;
      else
      {
         PTR=(node*)malloc(sizeof(node));
         if(h==0)
         {
            PTR->value=v;
            PTR->next=NULL;
            h=PTR;
            q=PTR;

         }
         else
         {
            PTR->value=v;
            PTR->next=NULL;
            q->next=PTR;
            q=PTR;
         }
      }
      printf("\nEnter 1 for delete first/Enter 2 for delete last/Enter 3 for delete middle/Enter Others Key for Exit\n");
      scanf("%d",&t);
      if(t==1)
      {
         printf("After Deletion, The Linked List Values Are:\n");
         ptr=ptr->next;
         PTR=ptr;
         while(PTR!=NULL)
         {
            printf("%d\n",PTR->value);
            PTR=PTR->next;
         }
      }
      else if(t==2)
      {
         printf("After Deletion, The Linked List Values Are:\n");
         PTR=h;
         while(PTR->next!=NULL)
         {
            PTR1=PTR;
            PTR=PTR->next;
         }
         PTR1->next=NULL;
         PTR=h;
         while(PTR!=NULL)
         {
            printf("%d\n",PTR->value);
            PTR=PTR->next;
         }
      }
      else if(t==3)
      {
         printf("Enter the delete value:\n");
         scanf("%d",&delete_value);
```

```
    }
    printf("The Linked List Values Are:\n");
    PTR=h;
    ptr=h;
    while(PTR!=NULL)
    {
       printf("%d\n",PTR->value);
       PTR=PTR->next;

    }
    int t;
    for(;;)
    {
            printf("After Deletion, The Linked List Values Are:\n");
            PTR=h;
            while(PTR->value!=delete_value)
            {
               PTR1=PTR;
               PTR=PTR->next;
            }
            PTR1->next=PTR->next;
            PTR=h;
            while(PTR!=NULL)
            {
               printf("%d\n",PTR->value);
               PTR=PTR->next;
            }
        }
        else
           break;
      }
}
```

Sample Output:

```
Enter your value (-99 to break)
10
Enter your value (-99 to break)
20
Enter your value (-99 to break)
30
Enter your value (-99 to break)
-99
The Linked List Values Are:
10
20
30

Enter 1 for delete first/Enter 2 for delete last/Enter 3 for delete middle/Enter Others Key for Exit
2
After Deletion, The Linked List Values Are:
10
20

Enter 1 for delete first/Enter 2 for delete last/Enter 3 for delete middle/Enter Others Key for Exit
```

Practice Problems

I. Write a program to delete all nodes containing a specific value.
II. Write a program to delete a linked list completely (free all nodes).

Week 13

A program for **inserting** nodes into a binary search tree.

Program

```
#include<stdio.h>
struct mynode
{
   struct mynode *ll;
   int d;
   struct mynode *rl;
};
typedef struct mynode node;
int main()
{
   node *t,*p,*q,*a[100];
   int i,j,x,n;
   t=0,n=0;
   for( ; ;)
   {
      printf("Enter your value (negative value to break)\n");
      scanf("%d",&x);
      if(x<0)
         break;
      q=(node*)(malloc(sizeof(node )));
      printf("q=%x\n",q);
      q->d=x;
      q->ll=0;
      q->rl=0;
      if(t==0)
         t=q;
      else
      {
         p=t;
         for( ; ;)
         {
            if(p->d==x)
            {
       printf("Sorry duplicate value\n");
               break; }
else
            {
               if(p->d>x)
               {
                  if(p->ll==0)
                  {
                     p->ll=q;
                     break;
                  }
                  else
                     p=p->ll;
               }
               else
               {
                  if(p->rl==0)
                  {
                     p->rl=q;
                     break;
                  }

                  else
                     p=p->rl;
               }
            }
         }
      }
      n++;
   a[n]=q;
   }
   for(i=1; i<=n; i++)
{
   p=a[i];
   printf("%4x %4x %4d %4x\n",p,p->ll,p->d,p->rl);
}
}
```

Sample Output:

```
Enter your value (negative value to break)
34
q=b91450
Enter your value (negative value to break)
10
q=b91470
Enter your value (negative value to break)
57
q=b91490
Enter your value (negative value to break)
25
q=b914b0
Enter your value (negative value to break)
8
q=b914d0
Enter your value (negative value to break)
39
q=b914f0
Enter your value (negative value to break)
-100
b91450 b91470   34 b91490
b91470 b914d0   10 b914b0
b91490 b914f0   57    0
b914b0    0   25    0
b914d0    0    8    0
b914f0    0   39    0
```

Practice Problems

I. Write a program to find the minimum and maximum value in a BST.
II. Write a program to count total nodes and leaf nodes in a BST.

Week 14

A program for **Linear** and **Binary** Searching.

Program (Linear)

```
#include <stdio.h>
int main()
{
   int Search_Value, i, n;
   printf("Enter the number of elements in array\n");
   scanf("%d", &n);
   int A[n];
   printf("Enter %d integer value\n", n);
   for (i = 0; i < n; i++)
      scanf("%d",&A[i]);
   printf("Enter the search value\n");
   scanf("%d", &Search_Value);
   for (i=0; i<n; i++)
   {
      if (A[i] == Search_Value)
      {
         printf("%d is present at location %d\n", Search_Value, i);
         break;
      }
   }
   if (i == n)
      printf("%d isn't present in the array\n", Search_Value);
   return 0;
}
```

Sample Output:

```
Enter the number of elements in array
6
Enter 6 integer value
10 23 56 27 35 87
Enter the search value
27
27 is present at location 3
```

Program (Binary)

```
#include<stdio.h>
main()
{
   int i,j,n,sv,Found=-1,BEG, END, MID;
   printf("Enter the number of
element\n");
   scanf("%d",&n);
   int a[n];
   printf("Enter the value in ascending
order\n");
   for (i=0; i<n; i++)
      scanf("%d",&a[i]);
   printf("Enter the search value\n");
   scanf("%d",&sv);
   BEG=0;
   END=n-1;
   MID=(BEG+END)/2;
   while(BEG<=END)
   {
      if(a[MID]==sv)
      {
Found=1;
         break;
      }
      else if (BEG==END)
         break;
      else if (a[MID]>sv)
         END=MID-1;
      else
         BEG=MID+1;

      MID=(BEG+END)/2;
   }
   if(Found==1)
      printf("Value Found");
   else
      printf("Value does not found");
}
```

Sample Output:

```
Enter the number of element
6
Enter the value in ascending order
9 13 45 67 89 95
Enter the search value
89
Value Found
```

Practice Problems

I. Write a program to compare the execution time of linear and binary search for the same dataset.
II. Write a program to find all indices of a given value in an array using linear search.

Week 15

A program for different **sorting** operation (Insertion, Selection and Bubble).

Program (Insertion)

```
#include <stdio.h>
int main() {
   int n, i, j, key;
   printf("Enter the number of elements:\n");
   scanf("%d", &n);

   int A[n];
   printf("Enter %d elements:\n", n);
   for (i = 0; i < n; i++)
      scanf("%d", &A[i]);

   for (i = 1; i < n; i++) {
      key = A[i];
      j = i - 1;

      while (j >= 0 && A[j] > key) {
         A[j + 1] = A[j];
         j--;
      }
      A[j + 1] = key;
   }
   printf("Final Sorted array using Insertion Sort:\n");
   for (i = 0; i < n; i++)
      printf("%d ", A[i]);

   return 0;
}
```

Sample Output:

```
Enter the number of elements:
7
Enter 7 elements:
65 34 12 9 21 3 55
Final Sorted array using Insertion Sort:
3 9 12 21 34 55 65
```

Program (Selection)

```
#include <stdio.h>
int main() {
   int n, i, j, temp, min_idx;

   printf("Enter the number of elements:\n");
   scanf("%d", &n);

   int A[n];
   printf("Enter %d elements:\n", n);
   for (i = 0; i < n; i++)
      scanf("%d", &A[i]);

   for (i = 0; i < n - 1; i++) {
      min_idx = i;
      for (j = i + 1; j < n; j++) {
         if (A[j] < A[min_idx])
            min_idx = j;
      }
      temp = A[min_idx];
      A[min_idx] = A[i];
      A[i] = temp;
   }

   printf("Final Sorted array using Selection Sort:\n");
   for (i = 0; i < n; i++)
      printf("%d ", A[i]);

   return 0;
}
```

Sample Output:

```
Enter the number of elements:
6
Enter 6 elements:
98 67 6 23 44 87
Final Sorted array using Selection Sort:
6 23 44 67 87 98
```

Program (Bubble)

```
#include <stdio.h>
int main() {
   int n, i, j, temp;

   printf("Enter the number of elements:\n");
   scanf("%d", &n);

   int A[n];
   printf("Enter %d elements:\n", n);
   for (i = 0; i < n; i++)
      scanf("%d", &A[i]);

   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         if (A[j] > A[j + 1]) {
            temp = A[j];
            A[j] = A[j + 1];
            A[j + 1] = temp;
         }
      }
   }
   printf("Final Sorted array using Bubble Sort:\n");
   for (i = 0; i < n; i++)
      printf("%d ", A[i]);

   return 0;
}
```

Sample Output:

```
Enter the number of elements:
7
Enter 7 elements:
37 8 2 68 75 50 27
Final Sorted array using Bubble Sort:
2 8 27 37 50 68 75
```

Practice Problems

I. Write a program to sort an array in descending order using bubble sort.
II. Write a program to sort an array of strings using selection sort.

Final Challenge Problems

1. Implement a **menu-driven C program** to perform insertion, deletion, and display in both **stack and queue**.

2. Write a program **to search a node in BST recursively** and display its **depth/level**.

3. Write a program to **create and traverse a doubly linked list**.

4. Write a program to find the **intersection** of two linked lists.

5. Write a program to analyze a one-dimensional array and perform the following operations:

- Count the frequency of each unique element.
- Identify and display duplicate elements.
- Remove duplicates and display the new unique array.

Example:

Input: A = [5, 2, 5, 7, 2, 9]

Output:

- Frequencies: 5 → 2, 2 → 2, 7 → 1, 9 → 1
- Duplicates: 5, 2
- Unique Array: [5, 2, 7, 9]

The above problems are designed to evaluate students' comprehensive understanding of all topics covered throughout the course. These exercises integrate various data structure concepts and encourage analytical thinking, algorithmic design, and problem-solving skills.